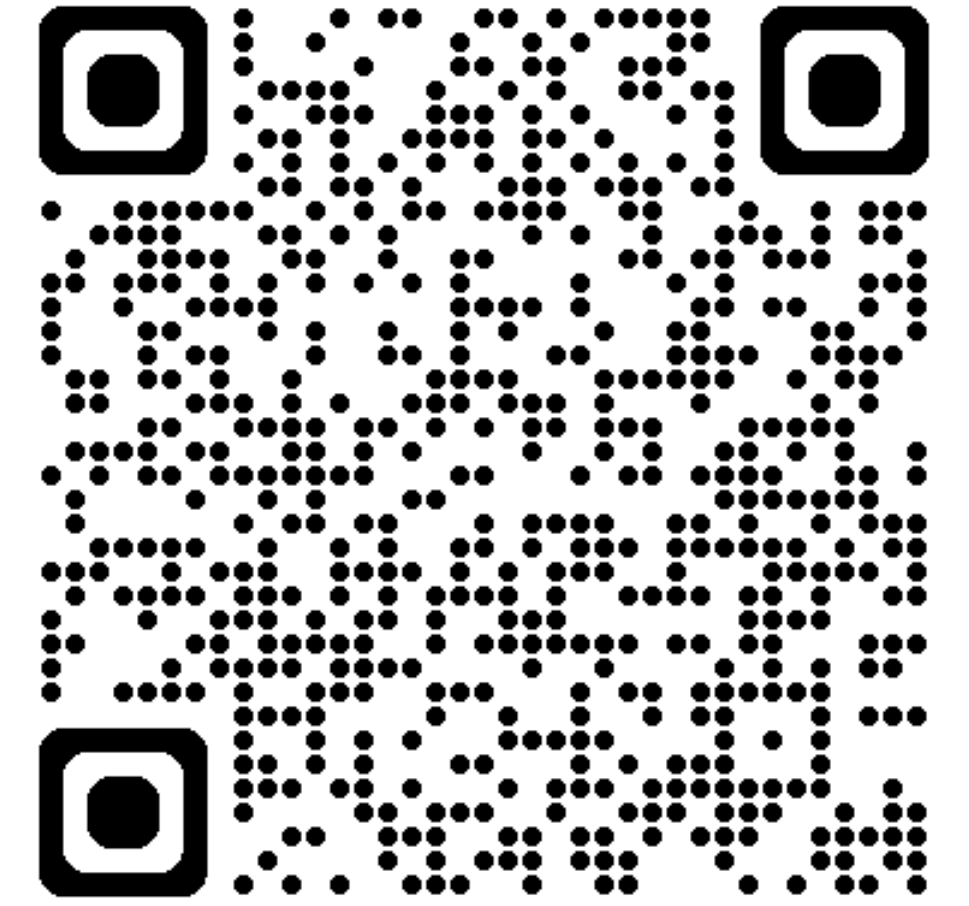


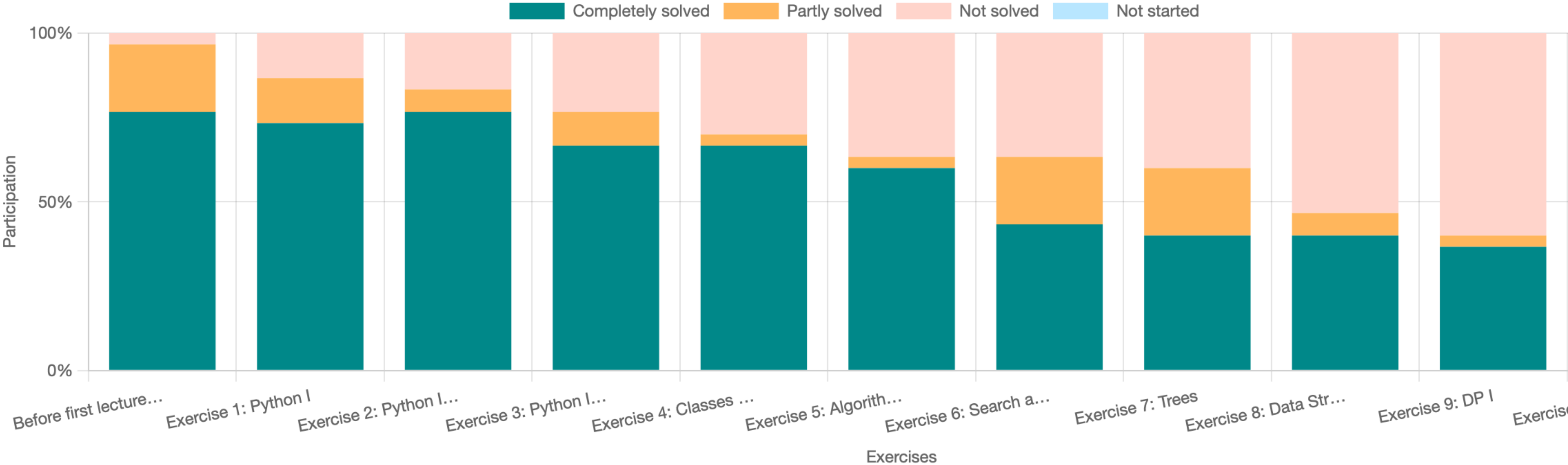
Übungsstunde 11



Heute:

- Repetitionsquiz
- Recap DP
- Weiter mit Dynamic Programming (in 2D)

Semester bald geschafft!



Rezept befolgen können

Kein Must! Aber wer keinen Anhaltspunkt hat eine sehr gute Guideline.

1. Identifiziere die Teilprobleme & Optionen
2. Definiere die Rekursion
3. Implementiere eine Lösung
 - Finde eine geeignete Datenstruktur
 - Identifiziere die Abhängigkeiten
 - Bestimme die Reihenfolge beim Ausfüllen der Struktur

Rezept befolgen können

Kein Must! Aber wer keinen Anhaltspunkt hat eine sehr gute Guideline.

Rezept Teilschritte

1. Identifiziere die Teilprobleme & Optionen
2. Definiere die Rekursion
3. Implementiere eine Lösung
 - Finde eine geeignete Datenstruktur
 - Identifiziere die Abhängigkeiten
 - Bestimme die Reihenfolge beim Ausfüllen der Struktur

Problemstellung

- **Eingabe:** Ein Schachbrett a als 2D-Array der Grösse $n \times m$, wobei jede Zelle eine nicht-negative ganze Zahl enthält, welche die entsprechende Belohnung angibt.
- **Ziel:** Der König startet auf Zelle $[0, 0]$ und kann sich in drei Richtungen bewegen: nord-östlich, östlich und süd-östlich. Das Ziel ist es, den optimalen Pfad zu finden, um die Gesamtbelohnung zu maximieren, während der König sich zur rechten Seite bewegt.
- **Ausgabe:** Die maximale Belohnung, die der König auf seinem Weg zur rechten Seite des Schachbretts einsammeln kann.

Aufgabe: Findet die jeweiligen Schritte im Rezept

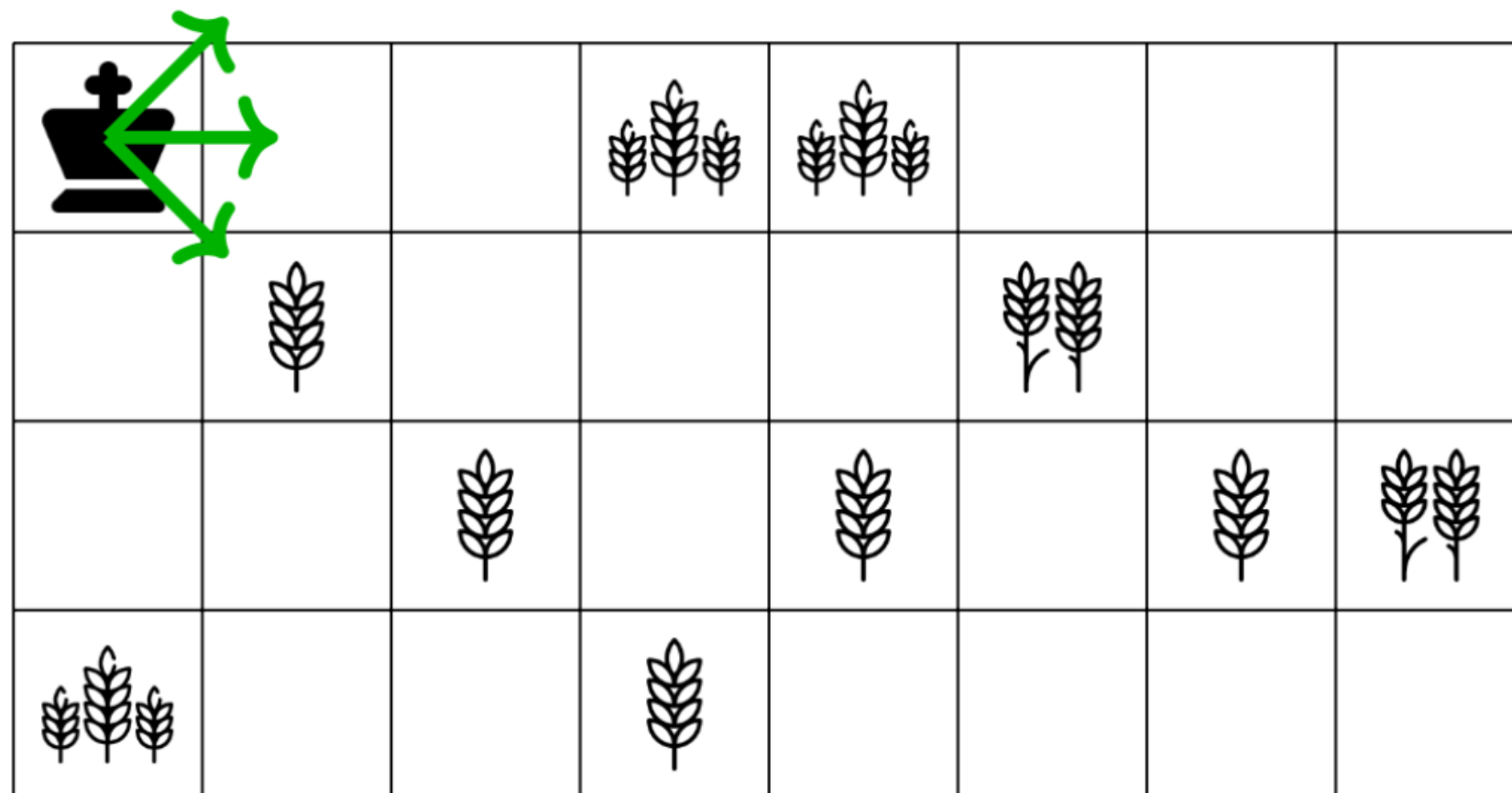
(Notizen von Vorteil)

Rezept befolgen können

Kein Must! Aber wer keinen Anhaltspunkt hat eine sehr gute Guideline.

Rezept Teilschritte

1. Identifiziere die Teilprobleme & Optionen
2. Definiere die Rekursion
3. Implementiere eine Lösung
 - Finde eine geeignete Datenstruktur
 - Identifiziere die Abhängigkeiten
 - Bestimme die Reihenfolge beim Ausfüllen der Struktur



Lösung

1) Teilprobleme: Anstatt von ganz links nach rechts zu kommen von $(m-1)$ zu m kommen. (Ist ein kleineres Problem)

Optionen: Nord-Ost, Ost, Süd-Ost

2) Rekursion definieren:

„Wie hängt ein Problem von seinen nächstkleineren Nachbarsproblemen ab“

=> Die Lösung einer Zelle ist:
Die Belohnung auf der Zelle selbst +
 $\max(\text{der Optionen})$

Rezept befolgen können

Rezept Teilschritte

1. Identifiziere die Teilprobleme & Optionen
 2. Definiere die Rekursion
 3. Implementiere eine Lösung
 - Finde eine geeignete Datenstruktur
 - Identifiziere die Abhängigkeiten
 - Bestimme die Reihenfolge beim Ausfüllen der Struktur
-
- 1) **Teilprobleme:** Anstatt von ganz links nach rechts zu kommen von $(m - 1)$ zu m kommen. (Ist ein kleineres Problem)
Optionen: Nord-Ost, Ost, Süd-Ost
 - 2) **Rekursion definieren:**
„Wie hängt ein Problem von seinen nächstkleineren Nachbarsproblemen ab“

=> Die Lösung einer Zelle ist:
Die Belohnung auf der Zelle selbst + max(der Optionen)

3.1) Datenstruktur: Wir wissen, dass unsere Teilprobleme in „2 Dimensionen wachsen“ => 2D-Tabelle. (Inf2 eigentlich immer 2d oder 1d Tabelle)

3.2) Abhängigkeiten: Siehe Rekursionsdefinition: wir müssen die Belohnung in der Zelle selbst kennen (bekannt)
Und die Lösungen der Zellen welche eine Option sind.

3.3) Daraus ergibt sich für die Reihenfolge:

von rechts nach links ausfüllen!

Repetitionsquiz

Erinnerung: Stab schneiden:

Aufgabe: mit papier/stift Pseudo-DP- Code ausdenken 3 Minuten, Zeile für Zeile überlegen.

Rückblick: Stab schneiden

mögliche Schritte zur Implementierung auf der nächsten Seite.

ich empfehle aber, solange es geht, ohne externen Input nachzudenken.

Aufgabenbeschreibung:

Ein Stab der Länge n muss in kleinere Stücke geschnitten werden. Jedes Stück der Länge i hat einen Preis $p[i]$, angegeben in einer Preisliste p .

- **Eingabe:** Länge n und Preisliste p .
- **Ziel:** Den Stab so zerschneiden, dass das Ergebnis den maximalen Wert hat.
- **Ausgabe:** Der maximale Wert, der durch das Zerschneiden des Stabes erzielt werden kann.

Drei Schritte der Dynamischen Programmierung

1. Identifiziere die Teilprobleme & Optionen
2. Definiere die Rekursion
3. Implementiere eine Lösung
 - Finde eine geeignete Datenstruktur
 - Identifiziere die Abhängigkeiten
 - Bestimme die Reihenfolge beim Ausfüllen der Struktur

mein Vorgehen

Stab schneiden Dynamic Programming

1. Datenstruktur überlegen & initialisieren;
DP inf 2, fast immer 1/2-Dimensionale Tabelle.
2. Nutzung von der Preistabelle überlegen. => in S integrieren und ggf. überschreiben.
3. Schleife, um die Tabelle von links nach rechts zu füllen.
4. Optionen sammeln, mit `max()` selektionieren

mein Vorgehen

Stab schneiden Dynamic Programming

ich habe meinen ausführlichen Gedankengang im .ipynb notiert.

Dies ist nicht zwingend ein gutes Beispiel von der Methodik her, kann aber helfen weiter Verständnis zu gewinnen.

wichtig für die folgenden Slides:

Ich habe in dieser Version mit zwei Zugriffen auf **S** (table im Code genannt) gearbeitet, in den folgenden Slides erfolgt ein Zugriff davon (der des Abschnittes) auf p.

Was ist eine optimale Teilstruktur?

„Der optimale Wert für einen Stab der Länge 5 ($S[5]$) hängt vom optimalen Wert für einen Stab der Länge 4 ($S[4]$) ab.“

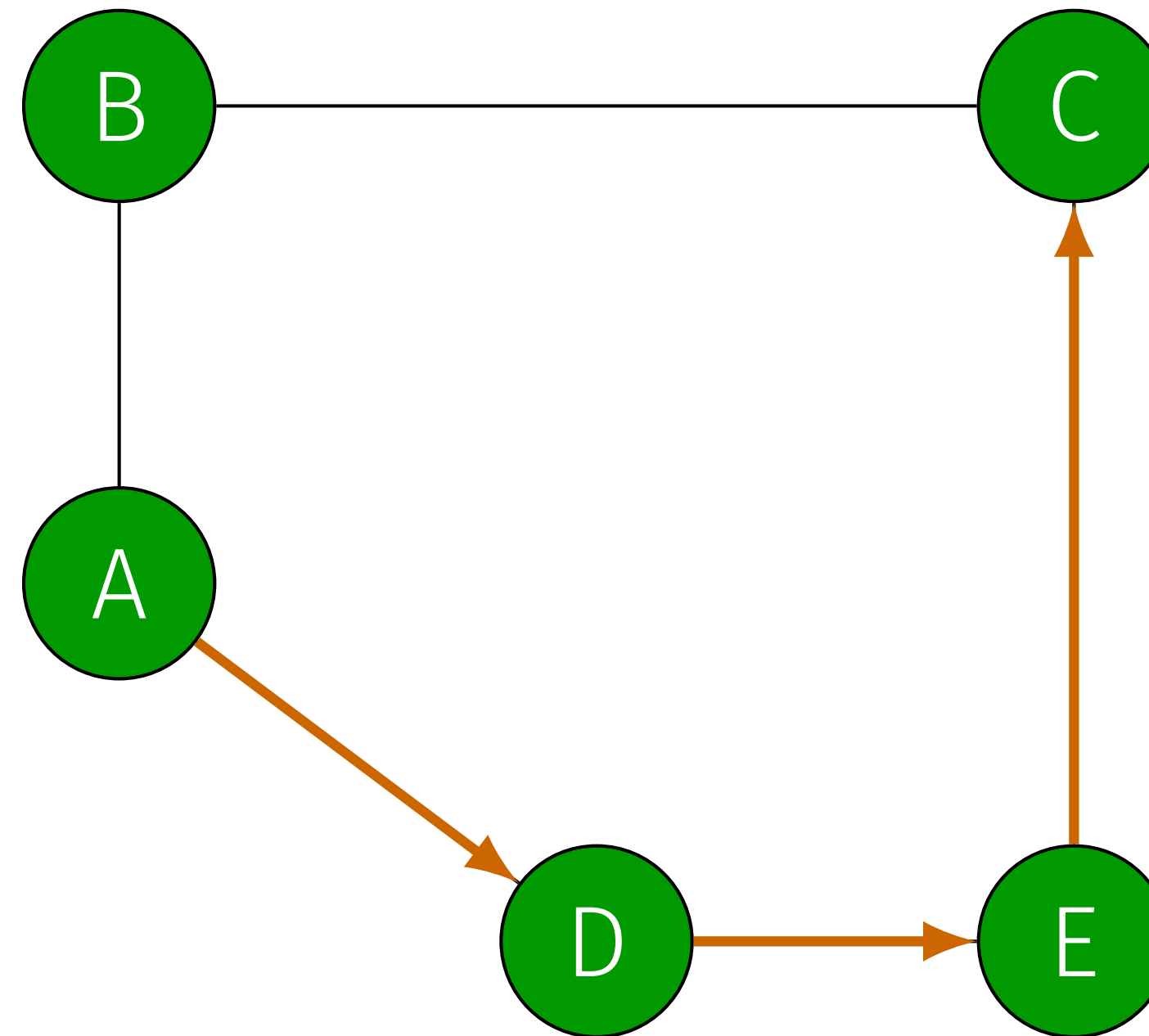
Beweis durch Widerspruch:

1. Wenn $S[5]$ optimal ist, muss $S[4]$ ebenfalls optimal sein.
2. Wäre $S[4]$ nicht optimal, könnte man einen höheren Wert für $S[4]$ erhalten.
3. Folglich könnte man auch einen höheren Wert für $S[5]$ finden. Daher kann $S[5]$ nicht optimal sein.

takeaway=> optimale Teilstruktur erlaubt das zusammensetzen der Lösungen von kleineren Problemen zur Lösung des grösseren Problems.

Beispiel: Nicht-Optimale Teilstruktur

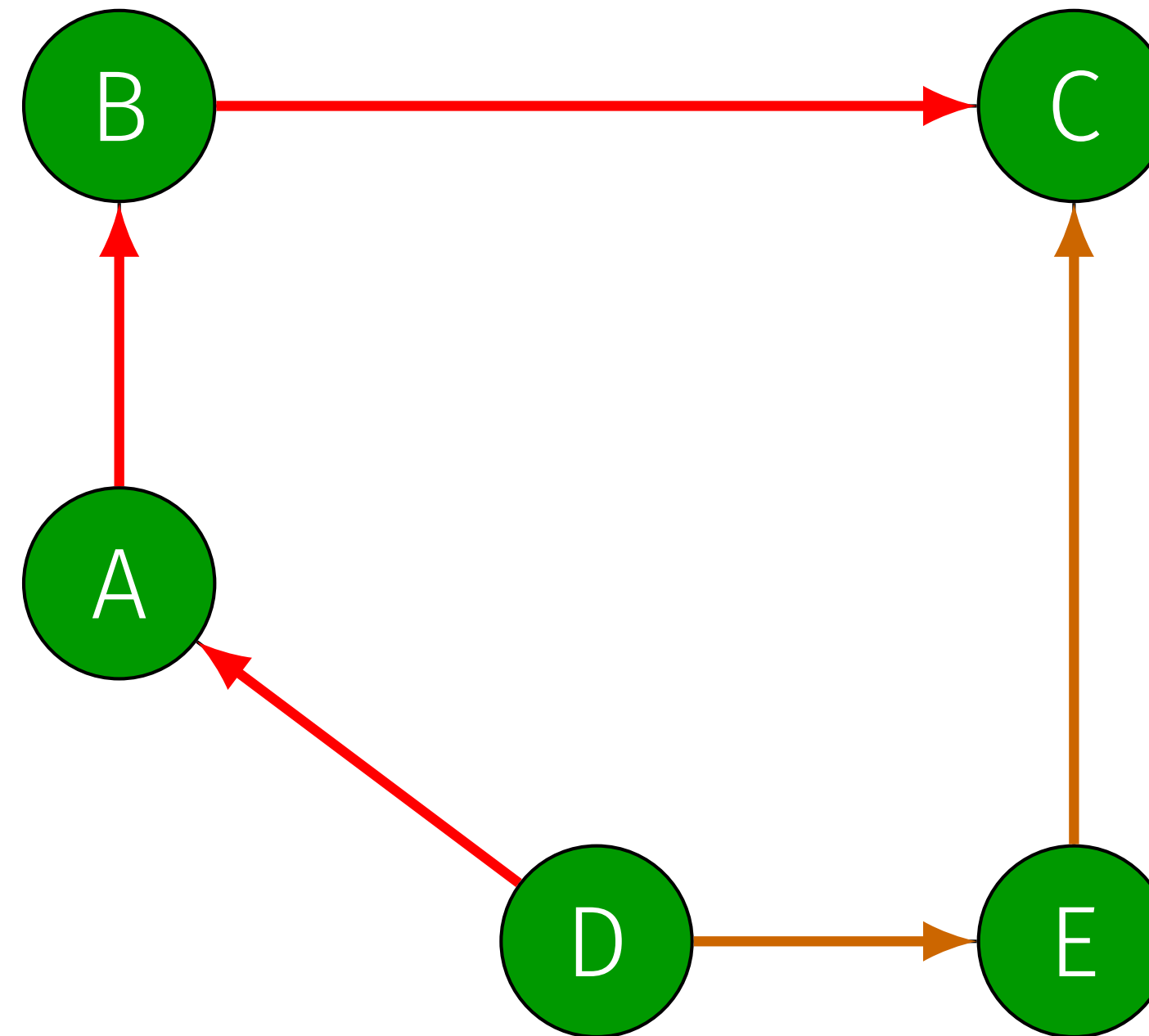
- Wir suchen den längsten geometrischen Pfad von A nach C.



Dieser Pfad kann in Pfade A–D und D–C aufgeteilt werden. (Teilprobleme)

Beispiel: Nicht-Optimale Teilstruktur

- Wir suchen den längsten geometrischen Pfad von A nach C.



Dieser Pfad kann in Pfade A–D und D–C aufgeteilt werden. (Teilprobleme)

- Der optimale (längste) Pfad von D nach C ist jedoch nicht Teil des optimalen Pfades von A nach C.

Rückblick: Stab schneiden

Warum können wir Dynamische Programmierung bei Stab schneiden verwenden?

1. Überlappende Teilprobleme

- Ohne Überlappungen würde ein DP-Algorithmus auch in exponentieller Zeit $\mathcal{O}(2^n)$ laufen.

→ DP führt nicht bedingungslos zu polynomieller Laufzeit.

(erlaubt es Lösungen mehrfach zu verwenden)

2. Optimale Teilstruktur

- Ohne optimale Teilstruktur kann die Lösung nicht mit DP erhalten werden.

Was bedeutet “begrenzt”?

Unbegrenzt:

- Der Stab kann so oft wie nötig geschnitten werden.

Begrenzt:

- Der Stab darf maximal c Mal geschnitten werden.

→ Der maximale Wert eines Stabes der Länge 20 unterscheidet sich je nachdem, ob er bis zu 10 Mal oder nur zweimal geschnitten werden darf.

sozusagen: Theorie vs Reallife mit einer Begrenzung an ressourcen.
Theoretischer Wert nicht immer erreichbar.

Drei Schritte der Dynamischen Programmierung

1. Identifiziere die Teilprobleme & Optionen
2. Definiere die Rekursion
3. Implementiere eine Lösung
 - Finde eine geeignete Datenstruktur
 - Identifiziere die Abhängigkeiten
 - Bestimme die Reihenfolge beim Ausfüllen der Struktur

so haben wir es auch Anfangs Lektion mit dem Stab gemacht

Schritt 1: Identifiziere die Teilprobleme & Optionen

Schritt 1: Identifiziere die Teilprobleme & Optionen

- $S[i]$ war der optimale Wert für einen Stab der Länge i .
- Jetzt hängt es auch davon ab, wie viele Schnitte noch übrig sind.
→ $S[i, 0], S[i, 1], S[i, 2], \dots, S[i, c]$

ein kleineres Teilproblem wäre ein stab der Länge l anstatt n ($l < n$) und s anstatt c verfügbar schnitte, ($s < c$)

weniger verfügbare Schnitte bedeuten weniger Komplexität, da weniger Entscheidungen gefällt werden müssen.

Schritt 1: Identifiziere die Teilprobleme & Optionen

- $S[i]$ war der optimale Wert für einen Stab der Länge i .
- Jetzt hängt es auch davon ab, wie viele Schnitte noch übrig sind.
→ $S[i, 0], S[i, 1], S[i, 2], \dots, S[i, c]$
- $S[i, t]$ bezeichnet den optimalen Wert, den man für einen Stab der Länge i mit t verbleibenden Schnitten erzielen kann.
→ **Zweidimensionale Teilprobleme!**

Schritt 1: Identifiziere die Teilprobleme & Optionen

Was sind die Optionen beim Teilproblem $S[i, t]$?

- Option 1: Schnitt nach Länge 1

$$S[i, t] = p[1] + S[i - 1, t - 1]$$

- Option 2: Schnitt nach Länge 2

$$S[i, t] = p[2] + S[i - 2, t - 1]$$

- Option 3: Schnitt nach Länge 3

$$S[i, t] = p[3] + S[i - 3, t - 1]$$

t-1! wir verlieren einen möglichen Schnitt.

Schritt 1: Identifiziere die Teilprobleme & Optionen

Was sind die Optionen beim Teilproblem $S[i, t]$?

- Option 1: Schnitt nach Länge 1

$$S[i, t] = p[1] + S[i - 1, t - 1]$$

- Option 2: Schnitt nach Länge 2

$$S[i, t] = p[2] + S[i - 2, t - 1]$$

- Option 3: Schnitt nach Länge 3

$$S[i, t] = p[3] + S[i - 3, t - 1]$$

- Option i : Schnitt nach Länge i (eg. Stück als ganzes verkaufen.)

$$S[i, t] = p[i] + S[0, t - 1]$$

Optionen

Keinen neuen Schnitt machen:

Wir behalten das Ergebnis, das wir bereits mit weniger Schnitten für diese Länge erzielt haben.

Einen neuen Schnitt machen:

Wir **schneiden ein Stück der Länge i ab**. Wir erhalten den Preis $P[i]$ und müssen den Reststab der Länge $L-i$ mit den verbleibenden $s-1$ Schnitten optimal verwerten.

Schritt 2: Definiere die Rekursion

tipp:

Keinen neuen Schnitt machen:

Wir behalten das Ergebnis, das wir bereits mit weniger Schnitten für diese Länge erzielt haben.

Einen neuen Schnitt machen:

Wir **schneiden ein Stück der Länge i ab**. Wir erhalten den Preis $P[i]$ und müssen den Reststab der Länge $L-i$ mit den verbleibenden $s-1$ Schnitten optimal verwerten.

im i stecken weitere optionen

Schritt 2: Definiere die Rekursion

- Aus allen Optionen wählen wir diejenige mit dem höchsten Wert.

$$S[i, t] = \max(p[k] + S[i - k, t - 1] \text{ for } k \text{ in range}(1, i + 1))$$

genau wie die Formel für unbegrenzte Schnitte nur diesmal
Sozusagen eine Spalte bei S in abhängigkeit der verfügbaren
Schnitte!

entweder Stab nicht schneiden ($k=i$), oder Länge i abschneiden.

Schritt 2: Definiere die Rekursion

Was war der Basisfall des unbegrenzten Stab-schneiden-Problems?

- $S[0] = 0$, ein Stab der Länge 0 hat keinen Wert.
→ Das ändert sich nicht, egal wie viele Schnitte noch übrig sind.

$$S[0, t] = 0, \quad t \in [0, c]$$

Was passiert, wenn wir keine Schnitte mehr haben?

- Nur der Wert des gesamten verbleibenden Stücks kann erzielt werden.

$$S[i, 0] = p[i], \quad i \in [0, n]$$

Schritt 2: Definiere die Rekursion

$$S[i, t] = \begin{cases} p[i] & \text{if } i = 0 \text{ or } t = 0 \\ \max(p[k] + S[i - k, t - 1] \text{ for } k \in [1, i]) & \text{else} \end{cases}$$

wie gesagt: bei keinem Schnitt-budget mehr, müssen wir den stab in der aktuellen länge verkaufen.

ansonsten können wir weiter auf optimale Teillösungen in unserer 2D Tabelle verweisen.

Schritt 3.1 - Wie lautet die optimale Datenstruktur?

Die Tabelle

ein Beispiel.

dazugehörige preisliste

```
#length:price
pricelist={
  0:0,
  1:1,
  2:5,
  3:8,
  4:9
}
```

Schnitte \ Länge	L 0	L 1	L 2	L 3	L 4	L 5	L 6
0 Schnitte	0	1	5	8	9	10	17
1 Schnitte	0	1	5	8	10	13	17
2 Schnitte	0	1	5	8	10	13	17
3 Schnitte	0	1	5	8	10	13	17

Die Tabelle

dazugehörige preisliste

```
#length:price
pricelist={
  0:0,
  1:1,
  2:5,
  3:8,
  4:9
}
```

ein Beispiel.

Schnitte \ Länge	L 0	L 1	L 2	L 3	L 4	L 5	L 6
0 Schnitte	0	1	5	8	9	10	17
1 Schnitte	0	1	5	8	10	13	17
2 Schnitte	0	1	5	8	10	13	17
3 Schnitte	0	1	5	8	10	13	17

Beachte: Ohne Schnitt verlieren wir hier, da eigentlich optimal 2x Länge 2 optimal wäre.

Abhängigkeiten/Reihenfolge des Auffüllens?

Abhängigkeiten/Reihenfolge des auffüllens?

Ein Blick auf die Rekursion:

$$S[i, t] = \max(p[k] + S[i - k, t - 1] \text{ for } k \text{ in range}(1, i + 1))$$

t-1 => von oben nach unten

**S(i-k) für k > 0 also: S(i-k) < S(i), wir hängen von kleineren Längen ab
=> links nach rechts**

Schritt 3: Implementiere eine Lösung

Wie müssen wir die Tabelle also auffüllen?

Schnitte \ Länge	L 0	L 1	L 2	L 3	L 4	L 5	L 6
0 Schnitte	0	1	5	8	9	10	17
1 Schnitte	0	1	5	8	10	13	17
2 Schnitte	0	1	5	8	10	13	17
3 Schnitte	0	1	5	8	10	13	17

links nach rechts, oben nach unten

**weitere Aufgabe:
Froschsprünge**

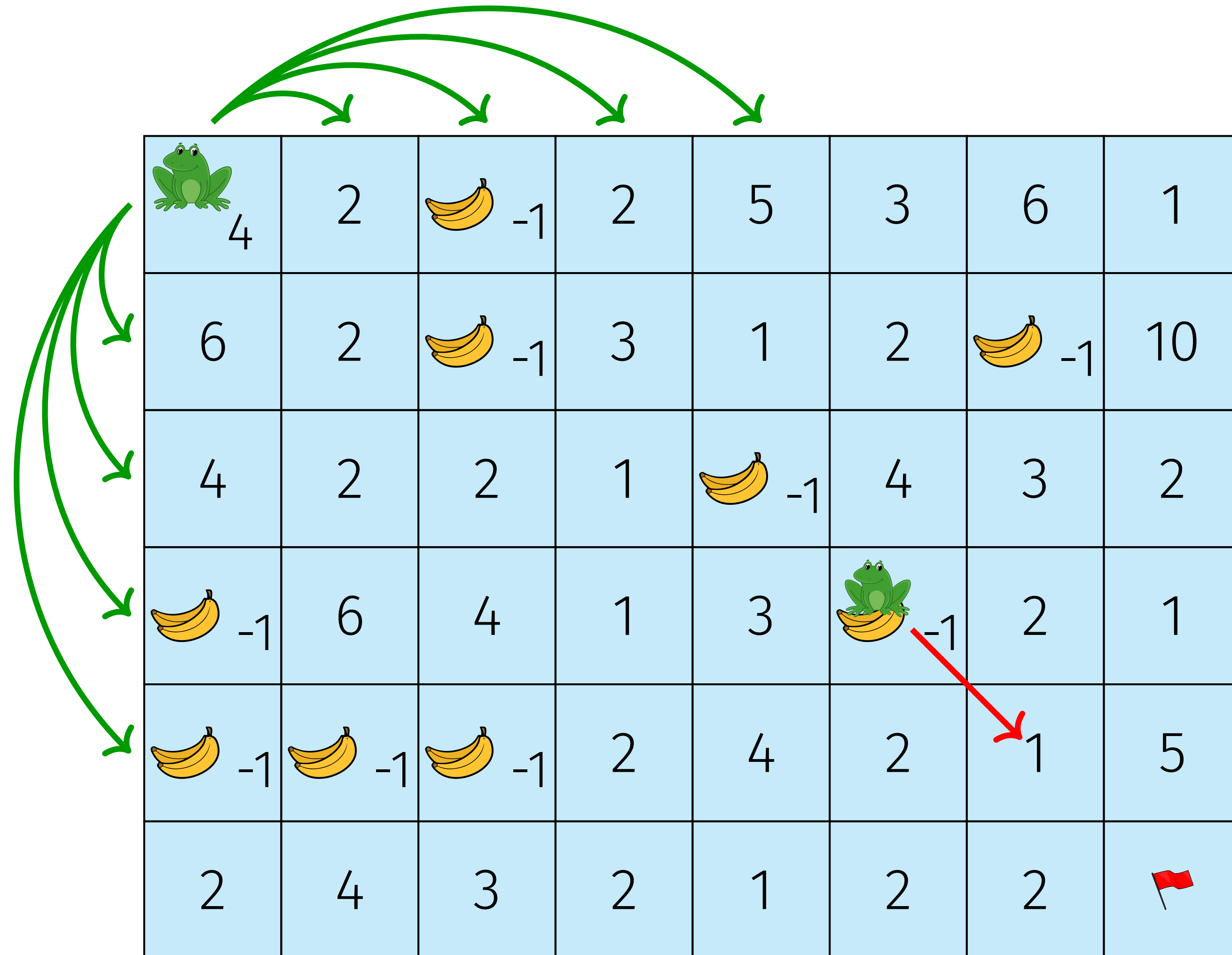
Aufgabenbeschreibung

- **Eingabe:** Ein 2D-Array a mit ganzen Zahlen aus der Menge $\{-1\} \cup [1, \infty)$.
- **Ziel:** Ein Frosch startet in der oberen linken Ecke und muss die untere rechte Ecke mit möglichst wenigen Sprüngen erreichen. Die geltenden Regeln werden auf der folgenden Folie erläutert.
- **Ausgabe:** Die minimale Anzahl an Sprüngen, die benötigt wird.

Aufgabenbeschreibung: Regeln

- Freies Feld ($a[i, j] > 0$):
 - Der Frosch kann entweder nach Osten oder nach Süden springen. Er kann eine Distanz von $1, 2, 3, \dots, a[i, j]$ Felder springen.
- Bananenschale ($a[i, j] = -1$):
 - Der Frosch rutscht auf der Bananenschale aus und landet ein Feld weiter in süd-östlicher Richtung (diagonal).
 - Das Ausrutschen auf einer Bananenschale zählt nicht als Sprung.

Aufgabenbeschreibung: Beispiel



Problemzerlegung

Dieses Problem ist sehr komplex. Es kann jedoch in Teilprobleme aufgeteilt werden, die unabhängig voneinander gelöst und kombiniert werden können.

1. Basisproblem
2. Variable Sprungdistanz
3. Zweidimensionalität
4. Bananenschale

Basisproblem

- **Eingabe:** Ein 1D-Array a mit positiven ganzen Zahlen ($a[j] > 0$).
- **Ziel:** Der Frosch muss Index $n - 1$ von Index 0 aus mit möglichst wenigen Sprüngen erreichen. Er hat zwei Optionen:
 - Er kann ein einzelnes Feld springen.
 - Er kann so viele Felder springen, wie der Wert des aktuellen Feldes $a[j]$ angibt.
- **Ausgabe:** Die minimale Anzahl an Sprüngen, um von links nach rechts zu gelangen.

Schritt 2: Definiere die Rekursion

$$S[j] = \begin{cases} 0 & \text{if } j = n - 1 \text{ (base case)} \\ 1 + \min(S[j + 1], S[j + a[j]]) & \text{else} \end{cases}$$

Randfall

Problem: Der Frosch springt über das Ziel hinaus.
→ Dies verursacht einen **IndexError: out of bounds**.

Wie können wir das lösen?

$$\min(j + a[j], n - 1)$$

- Dieser Ausdruck gibt $n - 1$ zurück, falls die Sprungdistanz ungültig ist.

Schritt 2: Definiere die Rekursion

$$S[j] = \begin{cases} 0 & \text{if } j = n - 1 \text{ (base case)} \\ 1 + \min(S[j + 1], S[j + a[j]]) & \text{else} \end{cases}$$

...wird zu...

$$S[j] = \begin{cases} 0 & \text{if } j = n - 1 \text{ (base case)} \\ 1 + \min(S[j + 1], S[\min(j + a[j], n - 1)]) & \text{else} \end{cases}$$

Findet: Datenstruktur und Abhängigkeiten für diese Teilaufgabe.

$$S[j] = \begin{cases} 0 & \text{if } j = n - 1 \text{ (base case)} \\ 1 + \min(S[j + 1], S[j + a[j]]) & \text{else} \end{cases}$$

...wird zu...

$$S[j] = \begin{cases} 0 & \text{if } j = n - 1 \text{ (base case)} \\ 1 + \min(S[j + 1], S[\min(j + a[j], n - 1)]) & \text{else} \end{cases}$$

Schritt 3: Implementiere eine Lösung

- Finde eine geeignete Datenstruktur:
 - 1D-Array, um die Teilprobleme $S[j]$ zu speichern.
- Identifiziere die Abhängigkeiten:
 - Um $S[j]$ zu berechnen, brauchen wir die Lösungen von $S[j + 1]$ und $S[j + a[j]]$.

Schritt 3: Implementiere eine Lösung

```
import numpy as np

def base_problem(a):
    n = len(a)

    S = np.zeros(n)
    S[-1] = 0 #Base case (redundant through np.zeros)

    for j in range(n-2, -1, -1):
        S[j] = 1 + min(S[j+1], S[min(j+a[j], n-1)])

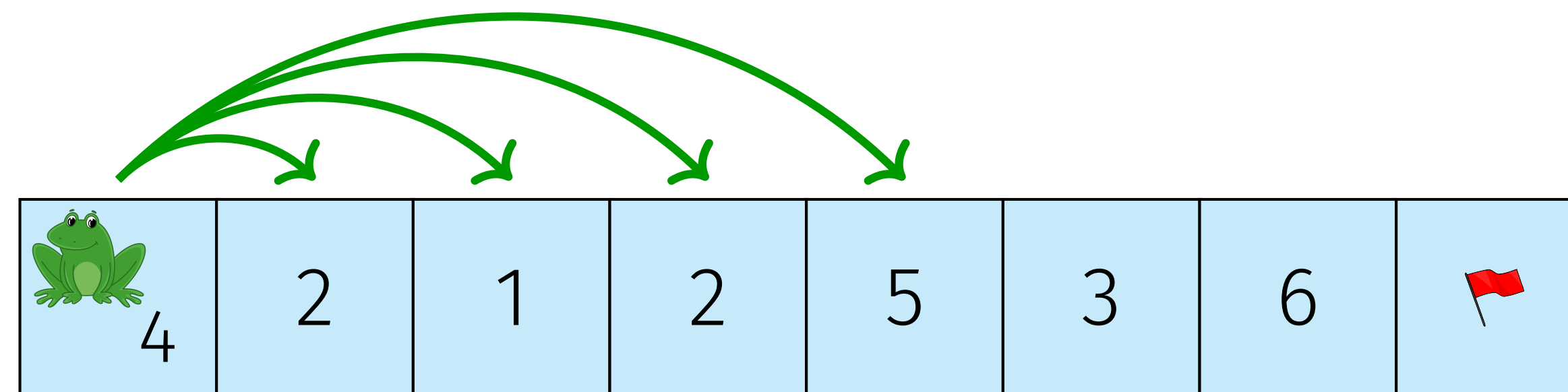
    return S[0]
```


Problemzerlegung

Nun fügen wir mehr Aspekte des originalen Problems hinzu.

1. Basisproblem
2. Variable Sprungdistanz
3. Zweidimensionalität
4. Bananenschale

Variable Sprungdistanz



- Jetzt kann der Frosch nicht nur ein oder $a[j]$ Felder springen, sondern jede Distanz von 1 bis $a[j]$ Felder.
- Wie müssen wir die bisherige Lösung verändern?

Schritt 2: Definiere die Rekursion

→ Wir prüfen alle Optionen mit einer for-Schleife.

$$S[j] = \begin{cases} 0 & \text{if } j = n - 1 \\ 1 + \min(S[j + 1], S[\min(j + a[j], n - 1)]) & \text{else} \end{cases}$$

ohne variable Sprunglänge (springt immer soweit es geht, was nicht immer optimal sein ist) (greedy)

⇓

$$S[j] = \begin{cases} 0 & \text{if } j = n - 1 \\ 1 + \min(S[\min(j + k, n - 1)] \text{ for } k \text{ in range}(1, a[j] + 1)) & \text{else} \end{cases}$$

Prüft alle möglichen Sprunglängen.

Schritt 2: Definiere die Rekursion

Wir können Iterationen sparen, indem wir den out-of-bounds-Vergleich in die `range` verschieben. Diese Änderung wird später weitere Vorteile haben.

$$S[j] = \begin{cases} 0 & \text{if } j = n - 1 \\ 1 + \min(S[\text{min}(j + k, n-1)] \text{ for } k \text{ in range}(1, a[j]+1)) & \text{else} \end{cases}$$

⇓

$$S[j] = \begin{cases} 0 & \text{if } j = n - 1 \\ 1 + \min(S[k] \text{ for } k \text{ in range}(j + 1, \text{min}(j + a[j], n - 1) + 1)) & \text{else} \end{cases}$$

range ende exklusiv

optmierung

Schritt 3: Implementiere eine Lösung

```
import numpy as np

def variable_jumping_distance(a):

    n = len(a)
    S = np.zeros(n)
    S[-1] = 0 #base case (redundant through np.zeros)

    for j in range(n-2, -1, -1):
        options = [S[k] for k in range(j+1, min(j+a[j], n-1)+1)]
        S[j] = 1 + min(options)

    return S[0]
```

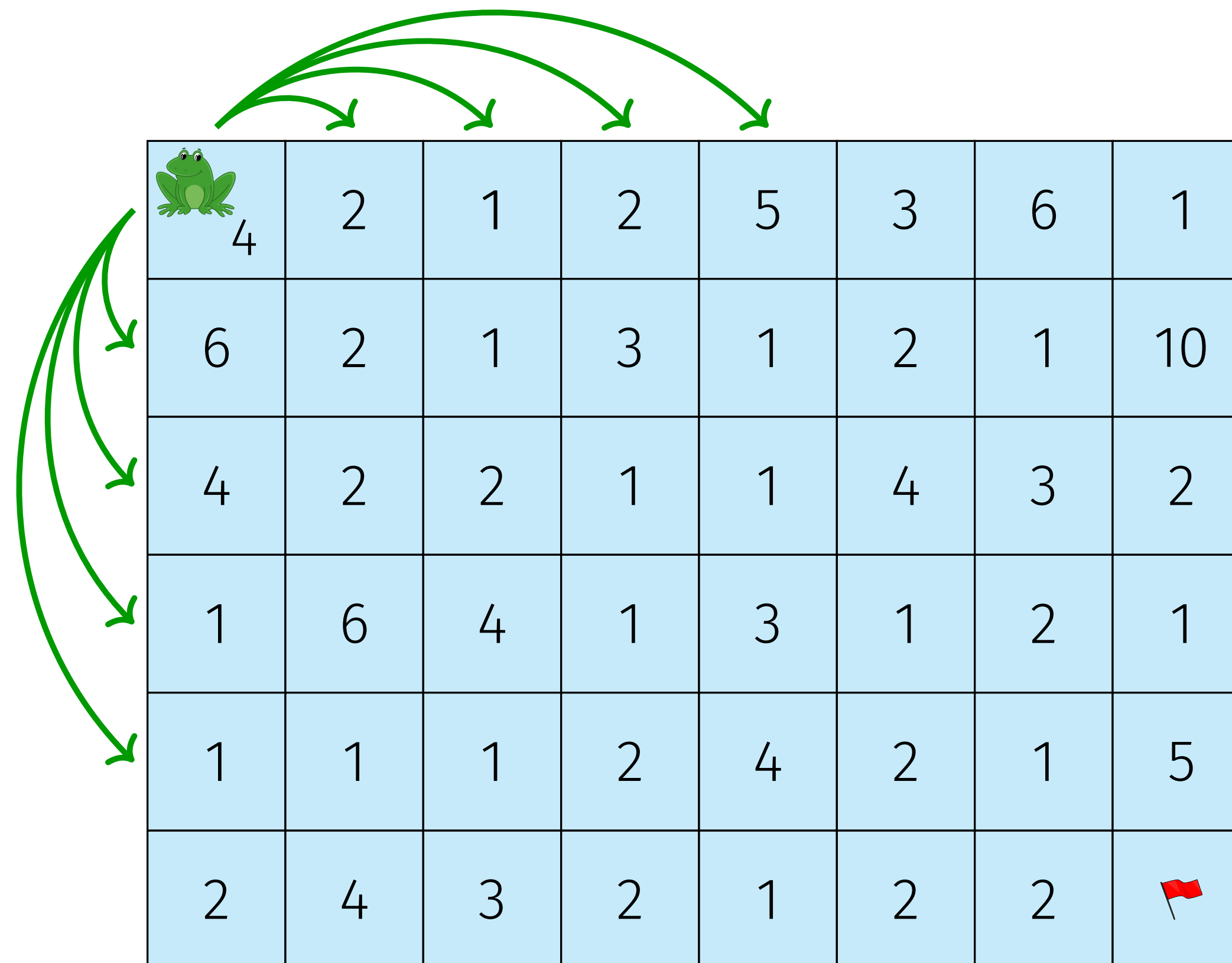

Problemzerlegung

Nun fügen wir mehr Aspekte des originalen Problems hinzu.

1. Basisproblem
2. Variable Sprungdistanz
3. Zweidimensionalität
4. Bananenschale

Zweidimensionalität

Jetzt ist die Eingabe ein 2D-Array. Der Frosch kann auch nach Süden springen.



Neu 2 Dimensionen, was bedeutet dies für unsere Optionen?

30sek. überlegen

Schritt 1: Identifiziere die Teilprobleme & Optionen

- Teilprobleme: unverändert. $S[i, j]$ bezeichnet die minimale Anzahl an Sprüngen, um von Position $[i, j]$ zum Ziel $[m - 1, n - 1]$ zu gelangen.
- Optionen: Mit der neu hinzugefügten Dimension kann der Frosch auch nach Süden springen. Das bedeutet, dass wir entscheiden müssen, ob wir bis zu $a[i, j]$ Felder nach unten oder nach rechts springen.
- Der bisherige Algorithmus wird kopiert für die neue Dimension. Anschliessend werden die beiden Ergebnisse verglichen.

Schritt 2: Definiere die Rekursion

→ Neu hinzugefügt wird der Vergleich zwischen Süd und Ost

$$S[i, j] = \begin{cases} 0 & \text{if } j = n - 1 \text{ and } i = m - 1 \\ 1 + \min(*south, *east) & \text{else} \end{cases}$$

mit...

$south = [S[k] \text{ for } k \text{ in range}(i + 1, \min(i + a[i, j], m - 1) + 1)]$

$east = [S[k] \text{ for } k \text{ in range}(j + 1, \min(j + a[i, j], n - 1) + 1)]$

Either the south or east list can be empty, but never both.

Todo: Datenstruktur, Abhängigkeiten&Reihenfolge.

Schritt 3: Implementiere eine Lösung

- Finde eine geeignete Datenstruktur:
 - Neue Dimension $\rightarrow$ ein 2D-Array mit der gleichen Größe wie das Eingabe-Array.
- Identifiziere die Abhängigkeiten:
 - Um $S[i, j]$ zu berechnen, müssen $a[i, j]$ Teilprobleme rechts und unterhalb gelöst sein. Für ein unbekanntes $a[i, j]$ heisst das: Alle Teilprobleme unterhalb und rechts.
- Bestimme die Reihenfolge beim Ausfüllen der Struktur:
 - Beginne bei $S[n - 1, m - 1]$ und fülle das Array von rechts nach links und unten nach oben aus.

Schritt 3: Implementiere eine Lösung

```
import numpy as np

def two_dimensionality(a):
    m,n = a.shape
    S = np.zeros((m,n))
    S[m-1,n-1] = 0 # base case (redundant through np.zeros)

    for i in range(m-1, -1, -1):
        for j in range(n-1, -1, -1):
            if (i == m - 1) and (j == n - 1):
                continue

            south = [S[k] for k in range(i+1, min(i+a[i,j], m-1) + 1)]
            east = [S[k] for k in range(j+1, min(j+a[i,j], n-1) + 1)]
            S[i,j] = 1 + min(*south, *east)

    return S[0,0]
```


Problemzerlegung

Alle vier Probleme werden kombiniert und das ursprüngliche Frosch-Sprünge-Problem wird gelöst.

1. Basisproblem
2. Variable Sprungdistanz
3. Zweidimensionalität
4. Bananenschale

Bananenschalen

- Im Eingabe-Array haben Bananenschalen den Wert -1 .
- Wenn der Frosch auf einer Bananenschale landet, rutscht er aus und fällt auf das nächste Feld in süd-östlicher Richtung.

$$S[i, j] \rightarrow S[i + 1, j + 1]$$

Bananenschalen

- Im Eingabe-Array haben Bananenschalen den Wert -1 .
- Wenn der Frosch auf einer Bananenschale landet, rutscht er aus und fällt auf das nächste Feld in süd-östlicher Richtung.

$$S[i, j] \rightarrow S[i + 1, j + 1]$$

- Auf einer Bananenschale ausrutschen zählt nicht als Sprung.
- Bananenschalen komme nicht an den Enden des Arrays $i = m - 1$ und $j = n - 1$ vor. Dort würde ausrutschen zu out-of-bounds-Zugriff führen.

Aufgabenbeschreibung: Erinnerung



Schritt 1: Identifiziere die Teilprobleme & Optionen

- Teilprobleme: unverändert. $S[i, j]$ bezeichnet die minimale Anzahl an Sprüngen, um von Position $[i, j]$ zum Ziel $[n - 1, m - 1]$ zu gelangen.
- Optionen: Nun hängen die Optionen davon ab, ob der Frosch auf einer Bananenschale landet.
- Im Fall, dass $a[i, j] = -1$, müssen wir nichts vergleichen. Wir übernehmen den Wert des Felds, auf welches der Frosch rutscht.

Schritt 2: Definiere die Rekursion

$$S[i, j] = \begin{cases} 0 & \text{if } j = n - 1 \text{ and } i = m - 1 \\ S[i + 1, j + 1] & \text{if } a[i, j] = -1 \text{ neu!} \\ 1 + \min(*south, *east) & \text{else} \end{cases}$$

mit *south*, *east* definiert wie bisher.

Schritt 3: Implementiere eine Lösung

- Finde eine geeignete Datenstruktur:
 - Unverändert, ein 2D-Array mit der gleichen Größe wie das Eingabe-Array.
Neu: Kann positive Ganzzahlen *oder* -1 enthalten.
- Identifiziere die Abhängigkeiten:
 - Unverändert
- Bestimme die Reihenfolge beim Ausfüllen der Struktur:
 - Unverändert

Problemzerlegung: Elemente

Das Basisproblem kann mit jeder Kombination von Teilproblemen ergänzt werden, die jeweils ein zusätzliches Element in die DP-Lösung einführen:

- Variable Sprungdistanz: Eine **zusätzliche for-Schleife** wird benötigt, da die Anzahl der Optionen nicht mehr konstant ist.
- Zweidimensionalität: Das Problem wird zweidimensional, wir verwenden die Variablen i und j .
- Bananenschalenfelder: Eine **zusätzliche if-Anweisung** wird benötigt, da sich die Optionen ändern können.

Kommentar

- Solche Extra-Komplexitäten wie eben variable Sprunglängen, oder Spezialfelder wie Bananenschalen lassen sich meist durch simple Tricks implementieren (extra For loop oder sonder Case/Option)
- hier hilft üben! Möglichst viele verschiedene Fälle gesehen zu haben.
- ein weiterer Trick: **Verbote** wie bsp. Hindernisse in einem Weg lassen sich oft mit einer unendlich schlechten Performance/kosten implementieren. tipp: **np.inf** generiert euch einen unendlich grossen Wert.

Exercise 10: DP II

Bemerkungen& Tipps

- **Alle empfohlen, auch in der Lernphase wieder anzuschauen!**
- **(für Lernphase: unter Code Examples DP Aufgaben weitere Aufgaben)**
- **Tipp für Mission Mars with Lava: np.inf**